

DK MEUBLE

Guide de déploiement production — VPS OVHcloud Rapport d'analyse + procédures (Laravel API + Next.js)

Document généré à partir de l'analyse réelle du dépôt. Aucune modification du code métier n'a été effectuée pour produire ce guide. Remplacez MON-DOMAINE / IP_DU_VPS / PASSWORD_SECURISE par vos valeurs.

IMPORTANT — Architecture réelle détectée

Ce projet n'est PAS un Laravel monolithique classique avec Blade/Vite comme site public. C'est un monorepo : **backend/** = API Laravel (Sanctum) et **frontend/** = Next.js 14. Le chemin demandé `/var/www/ecommerce/public` correspond au **public** Laravel (API + storage). Le site vitrine tourne sur Node (port 3000) derrière Nginx. Le dépôt contient déjà `docker-compose.yml`, `nginx/` et `.env.example` orientés production (`dkmeuble.sn` / `api.dkmeuble.sn`).

1. Rapport d'analyse du projet

1.1 Versions & stack

Type	Monorepo e-commerce : API Laravel + frontend Next.js
Laravel	^13.17 (composer.json)
PHP requis	^8.3 — installer PHP 8.3 + FPM
Frontend	Next.js 14.2.5 + React 18.3 + Tailwind 3.4
Node.js	≥ 18 recommandé (local détecté : v23.x OK)
Build front	next build / next start — PAS Vite pour le site public
Vite	Présent dans backend/ (assets Laravel optionnels) — secondaire
Auth API	Laravel Sanctum + Socialite (Google / Facebook)
DB défaut .env.example	sqlite en local ; MySQL 8 en prod (docker-compose)
Sessions / Cache / Queue	database (Redis non obligatoire)
Stockage images	disk public → storage/app/public (+ storage:link)
Scheduler	OUI — hourly (promos, wishlist, paniers abandonnés)
Queues	OUI — QUEUE_CONNECTION=database + jobs ShouldQueue
Paieement	Méthodes configurables admin (pas de PSP hardcodé dans .env)
SMS	SMS_DRIVER=log twilio (+ TWILIO_*)
Docker déjà présent	nginx, frontend, backend, mysql, certbot

1.2 Structure du dépôt

```
dk-meuble/  
├── backend/      # Laravel API (public/ = racine PHP)  
├── frontend/     # Next.js (site + /admin)  
├── nginx/        # default.conf + default.ssl.conf  
├── scripts/      # init-letsencrypt, smoke-tests, backups  
├── docker-compose.yml  
├── .env.example  # variables prod monorepo  
├── MISE_EN_LIGNE.md  
└── DEPLOIEMENT_VPS_OVH_DK_MEUBLE.pdf (ce document)
```

1.3 Problèmes / points de vigilance production

- Ne pas pointer Nginx uniquement sur `/var/www/ecommerce/public` pour le site public : le front Next.js serait inaccessible.
- Deux noms d'hôte recommandés : `dkmeuble.sn` (Next) + `api.dkmeuble.sn` (Laravel).
- `NEXT_PUBLIC_*` est injecté au build Next — rebuild frontend après changement d'URL HTTPS.
- `FILESYSTEM_DISK=public` + `php artisan storage:link` obligatoires pour images produits.
- Queues database : lancer un worker Supervisor (sinon jobs notifications/promos restent en attente).
- Scheduler : `cron * * * * * schedule:run` obligatoire.
- Ne jamais `migrate:fresh` / `APP_DEBUG=true` / `commit` `.env`.
- `CORS` + `SANCTUM_STATEFUL_DOMAINS` doivent matcher le domaine HTTPS du front.
- Redis : présent dans `.env.example` mais NON requis si cache/queue/session restent en database.

1.4 Services externes (.env)

MySQL	DB_HOST, DB_DATABASE, DB_USERNAME, DB_PASSWORD
URLs	APP_URL, FRONTEND_URL, NEXT_PUBLIC_API_URL, NEXT_PUBLIC_SITE_URL
Sanctum / CORS	SANCTUM_STATEFUL_DOMAINS, CORS_ALLOWED_ORIGIN
Admin seed	ADMIN_EMAIL, ADMIN_PASSWORD
Mail	MAIL_* (passer de log à smtp en prod)
SMS	SMS_DRIVER, TWILIO_SID, TWILIO_TOKEN, TWILIO_FROM
OAuth	GOOGLE_*, FACEBOOK_*, FRONTEND_URL
WhatsApp / téléphone	Admin → Paramètres (pas forcément .env)
AWS S3	Optionnel — AWS_* si vous quittez le disk local

2. Deux modes de déploiement possibles

Option A (recommandée, déjà dans le repo) — Docker Compose : Nginx + Next + Laravel + MySQL + Certbot. Moins de configuration manuelle PHP/FPM.

Option B (VPS nu Ubuntu) — Nginx + PHP 8.3-FPM + MySQL + Node + Supervisor + Certbot, avec chemins /var/www/dkmeuble/{backend,frontend}. Ci-dessous : procédures pour Option B alignées sur votre brief, adaptées au monorepo.

3. Préparation VPS Ubuntu (sécurité)

Objectif

Sécuriser le VPS : mises à jour, utilisateur sudo, SSH, UFW (22/80/443), Fail2ban. MySQL non exposé.

Commandes

```
sudo apt update && sudo apt upgrade -y
sudo adduser deploy
sudo usermod -aG sudo deploy
# Copier votre clé SSH, puis désactiver PasswordAuthentication dans /etc/ssh/sshd_config
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo apt install -y fail2ban
sudo systemctl enable --now fail2ban
```

Vérification

```
sudo ufw status
sudo systemctl status fail2ban
```

Résultat attendu : ports 22, 80, 443 autorisés ; Fail2ban actif.

DNS

Chez votre registrar : enregistrements A — MON-DOMAIN.COM → IP_DU_VPS ; www → IP_DU_VPS ; api.MON-DOMAIN.COM → IP_DU_VPS. Propager avant Certbot.

4. PHP 8.3 + extensions

Objectif : installer la version exigée par composer.json (php ^8.3) et les extensions Laravel.

```
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:ondrej/php
sudo apt update
sudo apt install -y php8.3-fpm php8.3-cli php8.3-mysql php8.3-curl php8.3-mbstring \
    php8.3-xml php8.3-zip php8.3-bcmath php8.3-gd php8.3-intl php8.3-sqlite3
php -v
php -m | egrep 'curl|mbstring|pdo_mysql|xml|zip|gd|bcmath|intl'
```

Résultat attendu : PHP 8.3.x CLI + FPM, modules listés présents.

Composer

```
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer
composer --version
```

Node.js (frontend Next.js)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v && npm -v
```

5. MySQL

```
sudo apt install -y mysql-server
sudo mysql_secure_installation
sudo mysql -e "
CREATE DATABASE ecommerce CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'ecommerce_user'@'localhost' IDENTIFIED BY 'PASSWORD_SECURISE';
GRANT ALL PRIVILEGES ON ecommerce.* TO 'ecommerce_user'@'localhost';
FLUSH PRIVILEGES;"
# Vérifier que bind-address = 127.0.0.1 (pas d'exposition publique)
```

Vérification : `mysql -u ecommerce_user -p ecommerce -e 'SELECT 1;'`

6. Déploiement du code (Git)

Objectif : cloner le monorepo. Nginx API → backend/public. Front → process Next.js.

```
sudo mkdir -p /var/www/dkmeuble
sudo chown -R deploy:deploy /var/www/dkmeuble
cd /var/www/dkmeuble
git clone https://github.com/VOTRE_ORG/dk-meuble.git .
# Structure : /var/www/dkmeuble/backend et /var/www/dkmeuble/frontend
```

6.1 Backend Laravel

```
cd /var/www/dkmeuble/backend
cp .env.example .env # puis éditer (voir §7)
composer install --no-dev --optimize-autoloader
php artisan key:generate --force
php artisan migrate --force
php artisan storage:link
php artisan config:cache
php artisan route:cache
php artisan view:cache
# NE PAS : php artisan migrate:fresh
```

Permissions

```
sudo chown -R deploy:www-data /var/www/dkmeuble/backend
sudo find /var/www/dkmeuble/backend/storage /var/www/dkmeuble/backend/bootstrap/cache -type d -
exec chmod 775 {} \;
sudo find /var/www/dkmeuble/backend/storage /var/www/dkmeuble/backend/bootstrap/cache -type f -
exec chmod 664 {} \;
# Éviter chmod -R 777
```

6.2 Frontend Next.js

```
cd /var/www/dkmeuble/frontend
# Créer .env.production ou exporter :
# NEXT_PUBLIC_API_URL=https://api.MON-DOMAIN.COM
# NEXT_PUBLIC_SITE_URL=https://MON-DOMAIN.COM
npm ci
npm run build
# Process manager (PM2) :
sudo npm i -g pm2
pm2 start npm --name dkmeuble-front -- start
pm2 save && pm2 startup
```

7. Fichier .env production (backend)

Ne jamais committer ce fichier. Exemple (valeurs fictives) :

```
APP_NAME="DK MEUBLE"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://api.MON-DOMAIN.COM
APP_KEY=base64:... # généré par artisan

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ecommerce
DB_USERNAME=ecommerce_user
DB_PASSWORD=PASSWORD_SECURISE

SESSION_DRIVER=database
CACHE_STORE=database
QUEUE_CONNECTION=database
FILESYSTEM_DISK=public

FRONTEND_URL=https://MON-DOMAIN.COM
SANCTUM_STATEFUL_DOMAINS=MON-DOMAIN.COM,www.MON-DOMAIN.COM
CORS_ALLOWED_ORIGIN=https://MON-DOMAIN.COM
```

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.votre-fournisseur.com
MAIL_PORT=587
MAIL_USERNAME=...
MAIL_PASSWORD=...
MAIL_FROM_ADDRESS=noreply@MON-DOMAINE.COM
MAIL_FROM_NAME="DK MEUBLE"

SMS_DRIVER=log
# SMS_DRIVER=twilio
# TWILIO_SID=...
# TWILIO_TOKEN=...
# TWILIO_FROM=...

# GOOGLE_CLIENT_ID=...
# GOOGLE_CLIENT_SECRET=...
# GOOGLE_REDIRECT_URI=https://api.MON-DOMAINE.COM/api/auth/oauth/google/callback
# FACEBOOK_CLIENT_ID=...
# FACEBOOK_CLIENT_SECRET=...
```

Redis : ne pas installer tant que QUEUE/CACHE/SESSION restent en database (configuration actuelle du projet / docker-compose).

8. Nginx

Objectif : front → proxy Next ; API → PHP-FPM sur backend/public ; bloquer .env/.git.

Fichier : /etc/nginx/sites-available/dkmeuble

```
# HTTP - ACME + redirect (après Certbot, adapter)
server {
    listen 80;
    server_name MON-DOMAINE.COM www.MON-DOMAINE.COM api.MON-DOMAINE.COM;
    location /.well-known/acme-challenge/ { root /var/www/certbot; }
    location / { return 301 https://$host$request_uri; }
}

# FRONT Next.js
server {
    listen 443 ssl http2;
    server_name MON-DOMAINE.COM www.MON-DOMAINE.COM;
    # ssl_certificate / etc. (Certbot)
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}

# API Laravel - SEUL public/ exposé
server {
    listen 443 ssl http2;
    server_name api.MON-DOMAINE.COM;
    root /var/www/dkmeuble/backend/public;
    index index.php;
    client_max_body_size 20M;

    location ^~ /storage/ {
        alias /var/www/dkmeuble/backend/storage/app/public;
        access_log off;
        expires 30d;
    }

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }
}
```

```

location ~ /\.php$ {
    include snippets/fastcgi-php.conf;
    fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
}

location ~ /\.(!well-known).* {
    deny all;
}
}

sudo ln -sf /etc/nginx/sites-available/dkmeuble /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

```

9. HTTPS (Certbot)

```

sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d MON-DOMAIN.COM -d www.MON-DOMAIN.COM -d api.MON-DOMAIN.COM
sudo certbot renew --dry-run

```

Résultat attendu : <https://MON-DOMAIN.COM> et <https://api.MON-DOMAIN.COM> OK.

10. Queues (Supervisor) + Scheduler (cron)

Nécessaire : jobs MonitorWishlistAlertsJob, CheckAbandonedCartsJob, MonitorPromoEndingJob, SendNotificationJob + Schedule hourly dans routes/console.php.

Fichier : /etc/supervisor/conf.d/dkmeuble-worker.conf

```

[program:dkmeuble-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/dkmeuble/backend/artisan queue:work database --sleep=3 --tries=3 --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=deploy
numprocs=1
redirect_stderr=true
stdout_logfile=/var/www/dkmeuble/backend/storage/logs/worker.log

sudo apt install -y supervisor
sudo supervisorctl reread && sudo supervisorctl update
sudo supervisorctl status
# Cron scheduler :
(crontab -u deploy -l 2>/dev/null; echo "* * * * * cd /var/www/dkmeuble/backend && php artisan
schedule:run >> /dev/null 2>&1") | crontab -u deploy -

```

11. Option Docker (déjà dans le dépôt)

Si vous préférez l'Option A : utiliser le docker-compose existant (nginx, frontend, backend, mysql, certbot).

```
cd /var/www/dkmeuble
cp .env.example .env
# Éditer APP_KEY, mots de passe MySQL, URLs https, CORS, ADMIN_PASSWORD
docker compose up -d --build
./scripts/init-letsencrypt.sh votre@email.com
# Rebuild front après HTTPS si NEXT_PUBLIC_* changent
```

12. Sauvegardes quotidiennes

```
# /usr/local/bin/backup-dkmeuble.sh
#!/bin/bash
set -euo pipefail
STAMP=$(date +%F)
DEST=/var/backups/dkmeuble/$STAMP
mkdir -p "$DEST"
mysqldump -u ecommerce_user -p'PASSWORD_SECURISE' ecommerce | gzip > "$DEST/db.sql.gz"
tar -czf "$DEST/storage.tar.gz" -C /var/www/dkmeuble/backend storage/app/public
# Copier .env hors dépôt (permissions 600) vers un stockage chiffré
find /var/backups/dkmeuble -mtime +14 -type d -exec rm -rf {} +

# cron : 0 3 * * * /usr/local/bin/backup-dkmeuble.sh
# Restauration DB : gunzip < db.sql.gz | mysql -u ... ecommerce
```

13. Logs & diagnostic

```
tail -f /var/www/dkmeuble/backend/storage/logs/laravel.log
tail -f /var/log/nginx/error.log
tail -f /var/log/php8.3-fpm.log
sudo supervisorctl tail -f dkmeuble-worker:dkmeuble-worker_00
pm2 logs dkmeuble-front
```

14. Mise à jour (git pull)

```
cd /var/www/dkmeuble
git pull
cd backend && composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:cache && php artisan route:cache
sudo supervisorctl restart dkmeuble-worker:*
cd ../frontend && npm ci && npm run build && pm2 restart dkmeuble-front
```

15. Tests de production

- <https://MON-DOMAINE.COM> — accueil Next
- <https://api.MON-DOMAINE.COM/api/footer> — JSON API
- Upload image produit → visible via /storage/...
- Admin login /admin/login
- Devis / contact → visibles en admin
- queue:work actif + schedule:run (promotions hourly)
- HTTP → HTTPS redirect
- APP_DEBUG=false

16. Optimisation e-commerce (sans changer le métier)

Le dépôt contient déjà une migration d'index performances (2026_09_13_120000_add_performance_indexes). En prod : config/route/view cache, images via storage public, pagination API, cache footer (FooterService). Ne pas activer Redis tant que non mesuré comme nécessaire.

Objectif final

Site : <https://MON-DOMAINE.COM> — API : <https://api.MON-DOMAINE.COM> — HTTPS, MySQL, images, auth, panier/commandes, queues + scheduler, sauvegardes, UFW.

Prochaine étape recommandée : choisir Option A (Docker) ou Option B (VPS nu), fournir le domaine réel + IP VPS, puis exécuter les commandes §3→§10 dans l'ordre.